Presentation Script — AI in FinTech: Credit Scorecard System

Format: Live demo on the **RiskLens** dashboard (not a PowerPoint).
Total length: 30 minutes · 6 speakers × 5 minutes each.
Backed by: `Final/fintech (1).pdf` (Tables 13 / 15 / 16) and `matlab/FinTech_Scorecard_Project.m`.

> **Before going live:**
> `cd fintech-dashboard && npm run dev` → open the dashboard, sit on **Slide 1 — Overview (/)**.
> Keep the report PDF open on a second device as backup.
> Use the numbered sidebar (1–11) to navigate — every speaker advances by clicking the next step.
>
> **Pacing note:** Each speaker has ~680 words of speech below — that lands on ~5:00 at a natural 130–140 wpm with the planned pauses. **Bracketed cues are silent** — they tell you where to point and click, not what to read.

Slide map (follow the numbered sidebar)

Step	Page	Speaker
1	Overview (`/`)	Speakers 1 & 2
2	WOE & Binning (`/woe`)	Speaker 3
3	Scorecard (`/scorecard`)	Speaker 3
4	ROC & Validation (`/validation`)	Speaker 4
5	Portfolio Risk (`/portfolio`)	Speaker 4
6	Accept / Reject (`/decisions`)	Speaker 5
7	Expected Loss (`/expected-loss`)	Speaker 5
8	Risk Pricing (`/pricing`)	Speaker 5
9	Model Performance (`/performance`)	Speaker 6
10	Final Decision (`/final`)	Speaker 6
11	Live Client Test (`/test`)	Speaker 6
12	MATLAB Charts (`/outputs/matlab`)	Optional appendix

Speaker 1 — Introduction, project framing & dashboard tour

Time: 5:00 · **Page:** Slide 1 — **Overview (/)**

[Stand near the screen. Dashboard already on Slide 1. Pause 2 seconds. Eye contact. Smile.]

Good morning everyone. The project we're presenting today is called **AI in FinTech — a Credit Scorecard System** — and what we want to show you is what happens when you take a real, messy business question that banks face every single day, and answer it with a transparent, data-driven tool.

The question itself is very simple. A bank receives a stack of loan applications. For every single applicant the bank has to make three decisions, all at the same time: **should we lend at all**, **how much money could we lose if this client defaults**, and **what is the minimum interest rate we have to charge so we don't lose money on average?** Get those three answers right consistently, and the bank makes a profit. Get them wrong, and it doesn't.

[Pause briefly. Open hand toward the screen.]

So instead of presenting our work to you in a static PowerPoint, we've built it into a fully interactive dashboard called **RiskLens**. Every chart, every metric, every table that you'll see today is **live**, generated from the **same MATLAB model and the same Excel data** that produced our written final report. There is no gap between what we say and what the system actually does — and that's deliberate, because **transparency** is the whole point of credit risk modelling.

[Brief pause.]

This is also why we framed the project as **AI in FinTech** rather than pure machine learning. In banking, you can't just hand a credit officer a black box and tell them to trust it — every decision has to be **explainable to the customer, to the auditor, and to the regulator**. So the model we built is deliberately simple on the surface and rigorous underneath. Four predictors. One score. One rule. One dashboard.

[Point at the badge in the top-left that reads "Executive Dashboard".]

The page we're on right now is the **Overview** — the executive summary of the entire project. Let me walk you through it.

[Point at the four metric cards along the top, left to right.]

Four headline numbers tell the whole story:

- **500 historical records** — these are past clients with **known outcomes**. We know whether each one paid back their loan or defaulted, and the model learns from them.

- **AUC equals 0.6775** — that's a standard measure of how well the model separates good clients from bad ones. We'll dig into what that number means later on; for now just remember it's the model's report card.
- **20 portfolio clients** — these are 20 brand-new applicants the bank wants us to evaluate. Once the model runs, **14 are accepted, 6 are rejected**.
- **$1.4 million of accepted exposure** with an **expected loss of $129,329.63** — that's the dollar amount the bank is putting at risk on the loans it would actually book, and the loss it should be ready to absorb.

[Pause for 2 seconds. Emphasize this result.]

So at a glance, the dashboard answers the bank's real question: *out of 20 people knocking on the door, who do we lend to, how much can we lose, and what should we charge them?* All in one place.

[Scroll slightly down so the "Key Parameters" panel on the right is visible.]

On the right of the page you can see the **ground rules** that apply across the whole project. Every loan is a **fixed $100,000**. If a client defaults, the bank still recovers **60%** of the money — through collateral, asset seizure, or collections — which leaves a **40%** loss per dollar lent. That 40% is called **LGD, Loss Given Default**. Our acceptance threshold is the **score 48.2032**, scores live between **0 and 100**, and the minimum interest rate is calculated as **PD times LGD**. You'll see every one of these numbers in action over the next twenty-five minutes.

[Hover briefly over the numbered sidebar on the left.]

Notice the **numbered sidebar** on the left — **steps 1 through 11**. That's the exact order of our presentation today. Each of my five teammates will take over for one or more of those steps, and we'll move through the dashboard live as if we were running this in front of a real credit committee. The flow is straightforward: we start with the data and the problem, build the model, validate it, apply it to twenty real applicants, and finish with the dollar outcomes and a live demo of the scoring engine.

[Pause for 2 seconds. Turn slightly toward the next speaker.]

To begin properly, my colleague will now explain **why this matters as a business problem** and walk you through the **data behind the model**.

> **Transition** → *"Let's start with the problem itself and the dataset behind everything you'll see today."*

Speaker 2 — Business problem, dataset & exploratory analysis

Time: 5:00 · **Page:** stays on Slide 1 — **Overview (/)**

[Take over the screen. Stay on Slide 1 — Overview. Look at the audience.]

Thank you. Before we get into models and graphs, I want to spend a minute on **why credit risk modelling is one of the most important problems in finance** — because it sets the context for everything we built.

When a bank lends money, it has **two ways to get the decision wrong**. If it accepts too many risky clients, defaults pile up and the bank loses real money. If it rejects too many safe clients, it loses revenue, market share, and reputation. The art of credit risk is finding the cutoff that minimises **both** mistakes at the same time. A **credit scorecard** is the tool the entire global banking industry uses to do that — it ranks applicants from safest to riskiest based on a handful of facts the bank already collects on every application form: **age, annual income, whether the client owns or rents their home, and whether they're formally employed**.

[Pause for 2 seconds.]

Our project uses exactly those four predictors — nothing exotic, nothing private — and the data we used comes from a single Excel file called `DataProjScoreCard.xlsx`, which contains two sheets.

[Point at the "Default Distribution — Historical Data" section.]

The first sheet is called **HistoricalData** — **500 past clients** the bank has already lent to. For each of them we know **age, income, residential status, employment status, and the default flag** — coded as **0 if they paid back normally** and **1 if they defaulted**.

[Point at the pie chart and the numbers next to it.]

Out of those 500 clients, **320 are good payers — the green slice — and 180 are defaulters — the red slice**. That gives us a portfolio default rate of **36%**.

[Pause. Emphasize this result.]

Thirty-six percent is **really high**. A typical retail bank works with a default rate between 3% and 8%. So either this is a stress-tested educational dataset or it represents a high-risk lending segment — either way, it's exactly the kind of dataset where a scorecard is most valuable, because every accept-or-reject decision really matters.

[Scroll down so the four "Default Rate by …" bar charts are visible.]

Now look at where the defaults are coming from. We split the 500 clients along each of our four predictors, and a very clear pattern appears every time.

[Point at “Default Rate by Income Band”.]

Income is the strongest single signal in the entire dataset. Clients earning **under $30,000** default about **57%** of the time. Clients earning **over $50,000** default only **21%**. That's almost a threefold gap — and it tells us immediately that **income** is going to be the heavyweight in our scorecard.

[Point at “Default Rate by Age Group”.]

Age tells a similar story. Clients under 35 default **50%** of the time. Clients above 65 only **17%**. There's a smooth, monotonic decline as people get older — older clients in this dataset are simply more reliable borrowers.

[Point at the residential and employment charts.]

Renters default at **40%** versus homeowners at **31%**. And on employment — clients labelled "**Other**" — the unemployed or self-employed — default at **44%**, against **28%** for fully employed clients.

[Pause. Look at the audience.]

So here's what's important: we're not inventing relationships out of thin air. The data itself is shouting at us — *people with low income, who are young, who rent, and who aren't formally employed are at much higher risk of default.* The model's job is just to put numbers on what we already see visually, and combine the four signals into a single, comparable score.

[Scroll down to the “MATLAB Workflow Pipeline” strip — do NOT read every step word-for-word.]

The pipeline behind everything you'll see today is shown here. In one sentence: we **load the data**, let MATLAB **group clients into risk buckets**, fit a **logistic regression**, **rescale into a 0–100 score**, **validate** on the same historical sample, and finally **apply the model to 20 new applicants** to make accept-reject decisions, calculate expected losses, and price each loan.

[Mention this business insight clearly.]

One last point before I hand over: the second sheet in our Excel file is called **ActualPortfolioData** — **20 brand-new applicants** the bank wants to evaluate. We don't know yet whether they'll default or not, because they haven't borrowed yet. That's the entire point of the model — to **predict** how they will behave, based on what we learned from the 500.

So we have **500 labelled past clients** and **20 unknown future clients**. The next stage is to translate those four features into a credit score.

> **Transition** → *"Now let's see how MATLAB groups the data and builds the actual scorecard."*

Speaker 3 — Binning, Weight of Evidence, Information Value & Scorecard build

Time: 5:00 · **Pages:** Slide 2 — **WOE & Binning (** `/woe` **)** then Slide 3 — **Scorecard (** `/scorecard` **)**

[Click step 2 in the sidebar — WOE & Binning. Wait for the page to load.]

We're now on **Slide 2 — WOE and Binning Analysis**. This is where the abstract data starts to take the shape of a scorecard.

[Point at the page subtitle: "Autobinning on 500 historical clients".]

The first step in MATLAB uses a function called **autobinning** from the Credit Scorecard Toolbox. Its job is to take each of our four predictors and **split clients into groups** in the smartest possible way — groups where the default rate inside one group is meaningfully different from the next group. Instead of using income as a raw dollar amount, the model uses it as **"is this client under $28,000, between $28 and $32k, $32 to $35k,"** and so on. That makes the model robust to outliers and easier to explain.

[Point at the four IV cards at the top of the page.]

Once the bins are formed, MATLAB calculates a number called **Information Value, or IV**, for each predictor. IV measures how much **separating power** a variable carries — the higher the IV, the more useful it is for predicting default.

Reading off the cards:

- **Income — IV equals 0.2355 — Strong** — our most powerful predictor.
- **Age — IV 0.1879 — Medium.**
- **Employment status — IV 0.1143 — Medium.**
- **Residential status — IV 0.0417 — Weak**, but still useful when combined with the others.

[Pause for 2 seconds. Mention this business insight clearly.]

The bank already learns something useful *before* we fit a single equation. Income matters more than housing. Employment matters more than housing. If the bank ever has to drop questions from its application form to speed things up, it now knows which ones it absolutely cannot cut.

[Scroll to the main WOE chart — Income should already be selected.]

The large chart below the cards shows **Weight of Evidence — WOE — for income**. Each bar is one income bin. **Green bars above the zero line** mean that bin is **safer than the portfolio average**. **Red bars below zero** mean **riskier than average**.

[Trace the bars from left to right with the cursor.]

The shape is almost perfect. Clients earning **under $28,000** sit deep in the red — a WOE of **–0.98**, the riskiest bin in the entire dataset. As income climbs, risk decreases smoothly, and by the time we reach **$50,000-plus** the WOE is **+0.72** — clearly safer than average. That smooth, monotonic curve from risky to safe is exactly what a logistic regression loves.

[Click the "Age" tab above the chart.]

Same story for **age**. Under 38 — red. Above 49 — strongly green, WOE around **+0.54**.

[Click "EmploymentStatus" then "ResidentialStatus".]

For employment, **Employed** is green, **Other** is red — a clean two-way split. For housing, **HomeOwner** is green, **renter** is red, but the gap is smaller, which matches its low IV.

[Scroll down to the four predictor cards with bin tables.]

Each card has a small table — the bin, the number of clients in it, the default percentage, and the WOE. These are exactly the values you'll find in **Tables 8 and 9 of our written report**.

[Click step 3 in the sidebar — Scorecard.]

We're now on **Slide 3 — Credit Scorecard Points**.

[Point at the four bar charts on the page.]

After WOE, MATLAB fits a **logistic regression** on all four predictors using the function `fitmodel`, and then calls `formatpoints` to **rescale everything into the 0-to-100 range** we want to show clients. The four charts here display the **points** each bin contributes.

[Point at the Income chart.]

Income dominates again — the **lowest** bin contributes **minus 8.68 points**, the **highest** contributes **plus 28.91 points**. That's a swing of nearly **38 points** from income alone.

[Point at the EmploymentStatus chart.]

Employment is the second strongest — Employed adds **+25.12**, Other only **+1.80**.

[Point at the ResidentialStatus and Age charts.]

Housing adds **+21.10** for owners versus **+6.71** for renters. And **age** adds from **+0.17** for the youngest bin all the way up to **+24.87** for the oldest.

[Emphasize this result.]

So a client's **final credit score** is just the sum: **age points plus income points plus housing points plus employment points**, clamped to 0–100. Higher score means lower risk. That's the entire scorecard — transparent, additive, and reproducible. The model is built. Now the question is: **does it actually work?**

> **Transition** → *"Let's validate the model on the historical data and find the right cutoff."*

Speaker 4 — Model validation (ROC, KS) and portfolio scoring (PD)

Time: 5:00 · **Pages:** Slide 4 — **ROC & Validation (** `/validation` **)** then Slide 5 — **Portfolio Risk (** `/portfolio` **)**

[Click step 4 in the sidebar — ROC & Validation.]

We're now on **Slide 4 — ROC Curve and Validation Metrics**. This page answers the question my teammate just left hanging: **how well does the scorecard actually work?**

[Point at the four metric cards at the top.]

We applied the MATLAB function `validatemodel` to the **same 500 historical clients**, and the dashboard shows the four official numbers — straight from **Table 13 of our report**:

- **AUC equals 0.6775**
- **KS Statistic equals 0.2892**
- **KS Optimal Score equals 48.2032**
- **Accuracy Ratio equals 0.3551**

Let me unpack what each of these actually means in plain English.

[Point at the ROC curve chart on the left. Trace the teal curve with the cursor.]

The **ROC curve** is the most important graph in the entire project. On the x-axis we have the **False Positive Rate** — the percentage of *good* clients we'd wrongly reject. On the y-axis we have the **True Positive Rate** — the percentage of *bad* clients we'd correctly catch. The **teal line is our model**. The **dashed grey diagonal is what you'd get from random guessing** — flipping a coin.

[Pause. Emphasize this result.]

Our curve sits **clearly above the diagonal across every single cutoff**. That means, no matter where we draw the line, our model catches **more defaulters than chance** while wrongly rejecting **fewer good clients than chance**. The model has **real predictive power**.

[Point at the AUC card.]

AUC — Area Under the Curve — is the area between the teal line and the bottom axis. **0.6775** means: if you pick one defaulter and one non-defaulter completely at random from the historical data, **there is roughly a 68% chance our model gives the defaulter the lower score**. For a four-variable model built on only 500 records, that is a solid, academically defensible result.

[Point at the KS card, then the KS Optimal Score card.]

KS — the Kolmogorov–Smirnov statistic — measures the **biggest gap** between the score distributions of good clients and bad clients. Our KS equals **0.2892**, and — this is the key — that

maximum gap occurs at score value **48.2032**. That number — **48.2032** — is the **single most important decision in the whole project**. It is our **acceptance threshold**, and we did not pick it from intuition. The data itself tells us this is the precise score at which good and bad clients are most cleanly separated.

[Read the rule slowly. Emphasize each word.]

> **Accept if the credit score is greater than or equal to 48.2032. Reject otherwise.**

[Point at the "Validation Summary" panel on the right, then pause.]

Right — we now have a working scorecard and a clear, data-driven rule. Time to apply it.

[Click step 5 in the sidebar — Portfolio Risk.]

We're now on **Slide 5 — Portfolio Scoring**. This is the moment we leave the past behind and apply the model to the **20 real applicants** from the `ActualPortfolioData` sheet.

[Point at the scatter chart.]

Every dot on this chart is one of the 20 applicants. The **x-axis is their credit score**. The **y-axis is their Probability of Default — PD** — calculated by the MATLAB function `probdefault`.

[Point at the red dashed vertical line at 48.2032.]

This red dashed vertical line is the **cutoff at 48.2032** — the exact KS optimal score we just discussed. Everything to the **right** of the line gets a loan. Everything to the **left** is rejected.

[Point at the cloud of dots — green vs red.]

Green dots are accepted, red dots are rejected. And notice the overall shape: the points fall consistently from upper-left to lower-right. **Higher score → lower PD**, with no exceptions. That **monotonic relationship** is exactly what you want from a sound credit scorecard.

[Hover over a green dot at the top of the green cluster — Client 4 at score 100.]

Our two safest applicants — clients **4 and 5** — have a perfect score of **100** and a PD of just **12.79%**.

[Hover over the lowest red dot — Client 1 at score 0.]

The riskiest applicant — Client 1 — has a **score of 0** and a PD of **70.59%**. Seven out of ten simulations on this client end in default.

[Scroll to the “Risk band segmentation” cards.]

We also tier all 20 applicants into **three risk bands**: **5 Low Risk, 9 Medium Risk, 6 High Risk** — and notice that all six in the high-risk band are exactly the six we rejected. The bands now give the bank a monitoring framework: low-risk clients get standard treatment; medium-risk clients get tighter limits; high-risk applicants don't get an offer at all.

> **Transition** → *"Now let's turn these scores into actual business outcomes — who gets the loan, how much we could lose, and what we should charge them."*

Speaker 5 — Decisions, Expected Loss & Risk-Based Pricing

Time: 5:00 · **Pages:** Slide 6 — **Decisions (`/decisions`)**, Slide 7 — **Expected Loss (`/expected-loss`)**, Slide 8 — **Pricing (`/pricing`)**

[Click step 6 in the sidebar — Accept / Reject.]

We're now on **Slide 6 — Accepted versus Rejected Clients**. This page **operationalises** the cutoff we just defined.

[Point at the "Decision Breakdown" bar chart on the left.]

The headline: applying the rule **score ≥ 48.2032** to all 20 applicants gives us **14 accepted** and **6 rejected** — a **70% acceptance rate**.

[Point at the "Client Scores vs Threshold" bar chart on the right.]

This chart sorts every client by score. **Green bars are accepted, red bars are rejected**, and the **dashed red line at 48.20 is the cutoff**.

[Point at the rightmost green bars — Clients 4 and 5.]

On the safe end you can see clients **4 and 5** with the maximum **score of 100**.

[Point at the borderline green bar — Client 8 at 55.84.]

And right at the cutoff — barely above the red line — is **Client 8 at score 55.84**. He's our most borderline accepted client — barely safe enough.

[Point at the leftmost red bar — Client 1.]

At the very bottom, Client 1 with a score of **0** — clearly out.

[Scroll down to the “Portfolio results table”.]

Now the full table — this is exactly **Table 15 of our report**, one row per client.

[Point at the column headers.]

For every client we have **ID, score, PD, decision, risk band, and expected loss in dollars**.

[Point at Client 4's row, then Client 1's row, then Client 8's.]

Three examples that tell the story:

- **Client 4** — score 100, PD 12.79%, accepted, Low Risk, expected loss **$5,114**. The cheapest, safest client.

- **Client 1** — score 0, PD 70.59%, rejected, High Risk, would have cost **$28,234** in expected loss. The clearest reject.
- **Client 8** — score 55.84, PD 33.50%, accepted, Medium Risk, expected loss **$13,402**. He's accepted, but he's the most expensive client we say yes to.

[Pause for 2 seconds. Mention this business insight clearly.]

So we're not just answering yes or no — the dashboard already tells the bank *which* of the accepted clients deserve closer monitoring.

[Click step 7 in the sidebar — Expected Loss.]

This is **Slide 7 — Expected Loss Dashboard**, where we roll the per-client losses into a single portfolio number.

[Point at the formula in the subtitle.]

The formula on top: **Expected Loss equals PD times LGD times Exposure**. In plain English: how likely they are to default, times how much we lose per dollar if they do — **40%** — times the loan size — **$100,000**.

[Point at the three metric cards.]

For the **14 accepted clients**, the system computes: **total Expected Loss $129,329.63**, **average PD 23.09%**, **accepted exposure $1,400,000**.

[Emphasize this result.]

This **$129,329.63** is the centrepiece of our results. It matches **Table 16** of the report and represents the dollar amount the bank should be ready to absorb on this $1.4 million loan book, assuming 60% recovery on defaults.

[Point at the "Expected Loss per Accepted Client" bar chart, then the tallest bar — Client 8.]

The bar chart underneath shows where the loss is concentrated. The tallest bar is — unsurprisingly — **Client 8, around $13,400**. He's accepted, but he is individually the largest single exposure.

[Point at the shortest bars — clients 4 and 5.]

The shortest bars are clients 4 and 5 at roughly **$5,100** each. Same loan size, very different expected loss — which is exactly what justifies the next step: **charging them different interest rates**.

[Click step 8 in the sidebar — Risk Pricing.]

This is **Slide 8 — Risk-Based Interest Rate Pricing**.

[Point at the formula in the subtitle.]

The pricing formula from our project brief: **minimum interest rate equals PD times LGD**. The rate has to **at least** cover the expected loss on that specific loan — otherwise the bank is lending at a guaranteed loss.

[Point at the bar chart and the gold dashed Avg line.]

Each bar is one client's minimum break-even rate, sorted from lowest to highest. Green is accepted, red is rejected. The gold dashed line is the **portfolio average across the 14 accepted clients — 9.24%**.

[Point at the leftmost green bars, then the rightmost.]

The safest clients — **4 and 5** — only need **5.11%** to break even. Our borderline client **8** needs **13.40%**. Still acceptable, but priced much higher to reflect the risk he carries.

[Point at the two summary boxes below.]

The two boxes underneath compress the whole pricing story: **lowest accepted rate 5.11%**, **highest rejected rate 28.23%** — that's Client 1, the score-zero applicant. To break even on him we'd need **28%** interest, which no responsible bank would ever charge a consumer. That is precisely why we reject him.

[Pause for 2 seconds. Mention this business insight clearly.]

This is what risk-based pricing means in practice: safer clients pay less, riskier clients pay more, and applicants who'd need an unethical rate to break even simply don't get a loan. Same scorecard, three completely different commercial outcomes.

> **Transition** → *"Let's see the consolidated performance, the final portfolio, and a live demo of the scorecard in action."*

Speaker 6 — Performance, Final Portfolio, Live Demo & Conclusion

Time: 5:00 · **Pages:** Slide 9 — **Performance (** `/performance` **)**, Slide 10 — **Final (** `/final` **)**, Slide 11 — **Test (** `/test` **)**

[Click step 9 in the sidebar — Model Performance.]

We're now on **Slide 9 — Model Performance Summary**. This page is a single-screen recap of the entire project.

[Point at the Validation Radar on the left.]

The radar on the left visualises our three validation metrics — **AUC, KS, and Accuracy Ratio** — all converted to percentages. The shape extends well beyond the centre, which tells us the model has **consistent, balanced predictive power** — not one lucky number.

[Point at the "Project Outcomes" list on the right.]

The list on the right is the **executive summary**: 500 historical observations, 36% historical default rate, AUC 0.6775, 14 of 20 portfolio clients accepted, 70% acceptance rate, 23.09% average PD on the accepted book.

[Mention this business insight clearly.]

The most important number on this page is the gap between **36% and 23%** — that's the **lift from using a scorecard**. We've moved the expected default rate from the full historical mix of **36%** down to **23.09%** on the clients we'd actually book. That's the business value of credit-risk modelling, expressed in a single sentence.

[Click step 10 in the sidebar — Final Decision.]

This is **Slide 10 — Final Portfolio Decision**. If we were standing in front of a real credit committee, this page is what we'd ask them to sign off on.

[Point at the gradient banner at the top — "14 Clients · $1.4M".]

The banner at the top is our final approved portfolio: **14 clients approved, $1.4 million in exposure, $129,329.63 of total expected loss, average PD 23.09%, average minimum interest rate 9.24%, and 6 clients rejected**.

[Point at the two columns underneath — Accepted on the left, Rejected on the right.]

The two columns underneath list every client by ID, with their score and PD. The 14 on the left are accepted, sorted by score; the 6 on the right are rejected — all with scores below 48.

[Pause for 2 seconds. Look at the audience.]

This is the **deliverable**. Now let's prove the scorecard works **live**, not just in a static table.

[Click step 11 in the sidebar — Live Client Test.]

This is **Slide 11 — Live Client Test**. You can pick any one of the 20 applicants — or even build a custom profile — and the system runs the full scorecard in real time.

[Open the dropdown. Pick "Client #4 — Score 100 (Accepted)".]

Let's start with our safest applicant — **Client 4**. The profile fills in instantly: **age 50, income $54,000, homeowner, employed**.

[Point at the "Scorecard point breakdown" table on the right.]

You can see his point breakdown live — Age +24.87, Income +23.84, Housing +21.10, Employment +25.12 — adding up to **94.94, capped at 100**.

[Click the teal "Run credit test" button.]

Now we click **Run credit test** — the dashboard runs the MATLAB pipeline, the gauge animates… and we get the result: **Score 100, PD 12.79%, Accepted, Low Risk, Expected Loss $5,114, Minimum Rate 5.11%**. **Exactly** the numbers in Table 15 of our report.

[Open the dropdown again. Pick "Client #1 — Score 0.0 (Rejected)". Click Run credit test.]

Now the opposite — **Client 1**. Score **0**, PD **70.59%**, **Rejected, High Risk**, expected loss **$28,234**, minimum rate **28.23%**. Same scorecard, opposite outcome — the model doing exactly what it should.

[Pause briefly. Mention this business insight clearly.]

In production this is the exact tool a credit officer uses — type the applicant's profile, click one button, get a transparent, defensible decision in under a second, with every piece of the score visible.

[Step back from the screen. Speak slowly. Eye contact.]

To conclude. We built a complete **AI-in-FinTech credit risk system** end to end — learning from **500 past clients**, validating with **AUC 0.6775**, accepting **14 of 20** new applicants at a data-driven cutoff of **48.2032**, with **$129,329.63** of expected loss on **$1.4 million** of exposure and rates from **5.11% to 13.40%**.

[Smile. Final eye contact.]

The real point: AI in finance is not about replacing the credit officer — it's about giving them a transparent, defensible tool where every decision ties back to four facts on a single application form. Thank you for your attention — we'd be happy to take your questions.

Q&A — short answers (any member can pick up)

Question	Suggested answer
Why train and validate on the same 500 clients?	The project brief specifies this approach. With only 500 records, splitting reduces sample size below what's reliable.
Is AUC 0.6775 "good"?	For a 4-variable scorecard, yes — defensible academically. A production bank model uses 30–50 variables and aims for AUC > 0.75.
Where does 48.2032 come from?	It's the score that maximizes the KS statistic — the biggest separation between good and bad clients on the historical data.
What's the difference between WOE and Points?	WOE is a raw log-ratio per bin. Points are WOE × regression coefficient, scaled to a 0–100 range.
Why is Client 8 accepted with a 13.40% rate?	His score (55.84) is above 48.20, so the model accepts him. The high rate reflects that he's our riskiest accepted client.
What happens if the economy changes?	This is the main limitation — the model is trained on past data. In production the bank would monitor PD vs realized defaults and recalibrate.
Could this be improved with more data?	Absolutely. Adding credit-bureau scores, debt-to-income ratios, and transaction history would lift AUC significantly.
Where is the code?	`matlab/FinTech_Scorecard_Project.m` and the `Final/` folder in the GitHub repository.

Speaker quick-reference (cheat card)

#	Speaker	Pages	Key numbers to mention
1	Intro & dashboard tour	Slide 1	500, 20, AUC 0.6775, 14 accepted, $1.4M, $129,329, 48.2032, $100k loan, LGD 40%
2	Problem, dataset, EDA	Slide 1	36%, 320/180, 57% / 21%, under-35 50%, renters 40%, Other 44%
3	WOE & Scorecard	Slides 2–3	IV: 0.2355 / 0.1879 / 0.1143 / 0.0417, points −8.68 to +28.91
4	Validation & Portfolio	Slides 4–5	AUC 0.6775, KS 0.2892, cutoff 48.2032, 5 / 9 / 6 risk bands
5	Decisions, EL, Pricing	Slides 6–8	14/6, $129,329.63, $1.4M, avg PD 23.09%, rates 5.11% – 28.23%, avg 9.24%
6	Performance, Final, Live demo	Slides 9–11	70% acceptance, 36% → 23%, Client 4 (100 / 12.79% / $5,114), Client 1 (0 / 70.59% / $28,234)

Rehearsal checklist

- [ ] Dashboard running and loaded on Slide 1 before the audience walks in
- [ ] Every speaker has clicked through their own slides at least once
- [ ] Client Test page tested with both Client 4 and Client 1
- [ ] Sidebar visible on the projection (not zoomed out)
- [ ] Backup report PDF open on a second device
- [ ] Internet not required — `npm run dev` runs locally
- [ ] **Time each speaker once** in rehearsal — aim for 5:00, accept 4:30–5:30

Script aligned with `Final/fintech (1).pdf` (Tables 13 / 15 / 16), `matlab/FinTech_Scorecard_Project.m`, and the RiskLens dashboard (`fintech-dashboard/src/config/presentationNav.ts`, steps 1–11).